

Assignment: Optimization and Decision Algorithms: From Pathfinding to Resource Allocation

林柏翰
1113405021
國立澎湖科技大學

Abstract—本報告聚焦於核心決策與最佳化演算法的理論分析與程式實作。實驗針對圖論中的最短路徑問題與資源分配中的經典挑戰，分別實作了廣度優先搜尋 (BFS)、Dijkstra 演算法、Bellman-Ford 演算法，以及 0/1 背包問題 (動態規劃)。報告中詳細記錄了各演算法之時間複雜度、核心設計邏輯與實際測試輸出結果，驗證了演算法在處理複雜網路結構與受限資源配置時的正確性與高效性。

Index Terms—Optimization Algorithms, Dijkstra, Bellman-Ford, BFS, Dynamic Programming, 0/1 Knapsack Problem, Java.

I. INTRODUCTION

在計算機科學與工程應用中，如何從眾多可行方案中尋求全域最佳解 (Global Optimal Solution) 是一項核心挑戰。最佳化決策演算法的應用範疇極其廣泛，從網路路由、地圖導航到金融資產配置皆能看見其蹤跡。本作業旨在透過 Java 程式語言，實作並深入分析四種經典的最佳化與走訪演算法：廣度優先搜尋 (BFS)、Dijkstra 最短路徑演算法、能處理負權重的 Bellman-Ford 演算法，以及利用動態規劃 (Dynamic Programming) 求解的 0/1 背包問題。透過終端機的實際輸出與時間複雜度分析，印證理論與實務的結合。

II. METHODOLOGY AND COMPUTATIONAL COMPLEXITY

本實驗實作的四種演算法架構與時間複雜度分析如下：

A. 廣度優先搜尋 (BFS)

BFS 是一種基於層級擴張的圖形走訪策略，適用於無權重圖形中尋找最少邊數的最短路徑。其核心機制為利用佇列 (Queue) 達成先進先出 (FIFO) 的同心圓式探索。其時間複雜度為 $O(V + E)$ ，其中 V 為節點數， E 為邊數。

B. Dijkstra 演算法

Dijkstra 演算法用於無負權重邊的加權圖形，尋找單源最短路徑。其採用貪婪策略 (Greedy Approach)，每次鎖定一個目前距離最小的節點，並對其相鄰節點進行鬆弛 (Relaxation) 更新。本實驗搭配最小堆疊優先佇列 (Priority Queue / Min-Heap) 進行最佳化，將尋找最小點的時間由 $O(V)$ 降至 $O(\log V)$ ，使得整體時間複雜度優化為 $O((V + E) \log V)$ 。

C. Bellman-Ford 演算法

當圖形中包含負權重邊時，Dijkstra 演算法的貪婪假設會失效。Bellman-Ford 演算法放棄了逐步鎖定節點的機制，改為對圖中所有的邊進行 $V - 1$ 輪的無差別全面鬆弛掃描，能完美包容負權重並偵測負權重循環 (Negative Cycles)。其時間複雜度為 $O(V \times E)$ 。

D. 0/1 背包問題 (動態規劃 DP)

0/1 背包問題為經典的組合最佳化與資源分配挑戰。本演算法將大問題拆分為小決策，利用二維網格狀態轉移表 $dp[i][w]$ 記憶過去的小解答，推導未來最優路線。其狀態轉移方程為：

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i) \quad (1)$$

其時間複雜度為 $O(N \times W)$ ，其中 N 為物品數量， W 為背包最大容量。

III. EXPERIMENTAL RESULTS AND ANALYSIS

透過 Java 程式碼實作，並輸入與簡報相對應的測試資料後，終端機的完整輸出結果如圖 1 所示：

A. BFS 走訪結果

於模擬的人際網路中，設定起點為 Marshall，終點為 May。程式運行後成功執行層級掃描，輸出之最短聯繫路徑為：

Marshall → Uriah → Andy → May

結果顯示總共經過 4 個節點，最少需要透過 2 個中間人，符合無權重圖形的最短路徑與最少人際跨度理論。

B. Dijkstra 演算法結果

以節點 1 作為走訪起點，計算至全圖各點之單源最短距離。終端機輸出結果如下：

- 到節點 1 的最短距離：0
- 到節點 2 的最短距離：4
- 到節點 3 的最短距離：6
- 到節點 4 的最短距離：9
- 到節點 5 的最短距離：5
- 到節點 6 的最短距離：11

各節點的距離鎖定與遞進過程精準展現了「降低 (Relax)」邊界更新的正確性。

C. Bellman-Ford 演算法結果

設定起點為節點 A，圖中包含一條負權重邊 $C \rightarrow B$ ：-50。經過 $V - 1$ 輪全面邊鬆弛掃描後，成功突破 Dijkstra 的盲區，輸出之全域最短距離為：

- 到節點 A 的最短距離：0
- 到節點 B 的最短距離：-40
- 到節點 C 的最短距離：10

由於路徑 $A \rightarrow C \rightarrow B$ 的總權重為 $10 + (-50) = -40$ ，比直接走 $A \rightarrow B$ 的距離 5 還短，結果證實演算法成功找出包含負權重的真正最短距離。

D. 0/1 背包問題結果

設定背包最大容量 $W = 40$ ，可選物品清單包含：物品 1 (重量 = 10, 價值 = 30)、物品 2 (重量 = 20, 價值 = 10)、物品 3 (重量 = 30, 價值 = 5)。經過動態規劃填表決策後，終端機輸出的最佳解為：

在不超過容量 40 的情況下，能裝入的最大總價值為：40

該結果為選擇物品 1 與物品 2 (總重量 $10 + 20 = 30 \leq 40$ ，總價值 $30 + 10 = 40$) 之組合，驗證了狀態轉移法則的正確性。

```
=====
1. 廣度優先搜尋 (BFS) - 尋找最少節點路徑
=====
[時間複雜度]  $O(V + E)$  (V: 節點數, E: 邊數)
[執行過程與結果]:
    目標: 在人際網路中尋找從 [Marshall] 到 [May] 的最短聯繫路徑。
    -> 成功找到路徑! 順序為: Marshall -> Uriah -> Andy -> May
    -> 總共經過節點數: 4 (需要透過 2 個中間人)

=====
2. Dijkstra 演算法 - 無負權重的單源最短路徑
=====
[時間複雜度]  $O((V + E) \log V)$  (使用 Priority Queue)
[執行過程與結果]:
    目標: 計算從起點 [1] 到圖中所有其他節點的最短距離。
    -> 到節點 1 的最短距離: 0
    -> 到節點 2 的最短距離: 4
    -> 到節點 3 的最短距離: 6
    -> 到節點 4 的最短距離: 9
    -> 到節點 5 的最短距離: 5
    -> 到節點 6 的最短距離: 11

=====
3. Bellman-Ford 演算法 - 容許負權重的最短路徑
=====
[時間複雜度]  $O(V * E)$  (需要執行  $V-1$  輪全圖掃描)
[執行過程與結果]:
    目標: 計算從起點 [A] 出發的最短距離 (包含負權重邊 C->B: -50)。
    -> 到節點 A 的最短距離: 0
    -> 到節點 B 的最短距離: -40
    -> 到節點 C 的最短距離: 10

=====
4. 0/1 背包問題 (動態規劃 DP)
=====
[時間複雜度]  $O(N * W)$  (N: 物品數量, W: 背包最大容量)
[執行過程與結果]:
    設定參數:
    -> 背包最大容量: 40
    -> 物品清單 (重量, 價值):
        物品 1: 重量=10, 價值=30
        物品 2: 重量=20, 價值=10
        物品 3: 重量=30, 價值=5
    -> [最佳解] 在不超過容量 40 的情況下，能裝入的最大總價值為: 40
=====
```

Fig. 1. 四種決策演算法於終端機之實際執行結果與時間複雜度輸出畫面。

IV. CONCLUSION

本次實驗透過完整的 Java 程式開發，深入探討了最佳化決策演算法的底層齒輪。實驗結果表明，BFS 與 Dijkstra 演算法能在各自對應的圖形結構中高效尋求路徑；Bellman-Ford 演算法提供了全局視野，完美包容了負權重邊；而動態規劃則透過記憶過去推導未來，成功解決了資源分配挑戰。本實驗為後續研究進階演算法與複雜系統最佳化奠定了扎實基礎。